Performance Analysis and Walltime Prediction for Neuroscience Applications

Axel Vivenot
axel.vivenot@telecom-paris.fr

Valentin Delis
valentin.delis@ensiie.fr

***Abstract*—Recent advancements in High-Performance Computing (HPC), Big Data, and Artificial Intelligence (AI) have driven an unprecedented demand for computational resources in neuroscience. However, efficiently utilizing these resources remains a challenge due to the complexity of modern computing systems and the diverse nature of neuroscience applications. This problem is exacerbated in neuroscience applications, where users are often not experts in computer science or HPC, making it difficult to optimize resource allocation and job scheduling. In this paper, we present a comprehensive performance analysis of several neuroscience applications, focusing on their execution time and memory footprint, and compare them across different system configurations. Finally, we develop a model to predict the execution time of one of these applications based on their input parameters and system configurations.**

***Index Terms*—Stochastic application, Execution time, Memory footprint**

I. Introduction

When designing an application, it is crucial to understand its performance consistency relative to input parameters. In the context of High-Performance Computing (HPC), resources are typically allocated based on walltime estimates. To schedule jobs efficiently and minimize wait times, users often provide an estimate of the expected execution time. Consequently, accurate prediction models are essential for efficient resource management and cost reduction. This prediction is dependent on a lot of factors, such as the input data, the system configuration, the hardware used, the software implementation, … This is particularly critical for stochastic applications where execution time varies significantly. With the increasing complexity of neuroscience applications driven by AI and Big Data, understanding these performance characteristics has become more important than ever.

In this paper, we present a comprehensive performance analysis of several neuroscience applications, focusing on characterizing their execution time and memory footprint relative to input data.

All experiments were conducted on a server node with the following specifications:

- CPUs: 2 AMD EPYC 7502 32-Core Processors (64 threads @ 2.50GHz)
- RAM: 504 GiB
- GPU: NVIDIA Quadro RTX 5000

To minimize interference, we ensured that no other significant user processes were active during experiments. Background system processes consumed approximately 1% of CPU and 6% of RAM (~30 GiB) on average.

We measured application memory usage using two methods:

- For Docker-based applications, we queried `/sys/fs/cgroup/memory.current` to obtain precise container memory usage.
- For native applications, we utilized `vmstat` to monitor system-wide memory fluctuations, as granular per-process tracking was not feasible for these specific workflows.

II. Spatially Localized Atlas Network Tiles (SLANT)

This study begins with a specific brain segmentation application called SLANT (Spatially Localized Atlas Network Tiles)[12, 13]. SLANT uses a fully convolutional network (FCN) to segment 3D MRI brain scans into different anatomical regions.

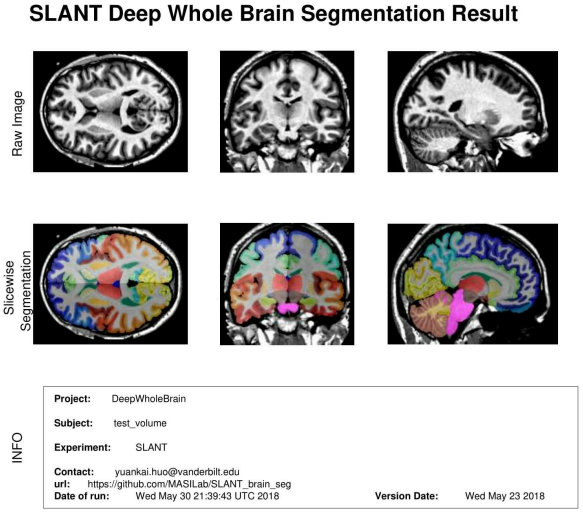

Fig. 1. SLANT output example

The segmentation is performed in three main steps: preprocessing, segmentation using a deep learning model, and postprocessing to generate the output result. SLANT exists in two versions: SLANT v1.0 and SLANT v1.1, with both versions supporting CPU and GPU modes. We will focus here on the overlapped version of SLANT: SLANT27, which uses 27 overlapped network tiles for segmentation.

A. Replicating Previous Results

In a 2020 study *Profiles of upcoming HPC Applications and their Impact on Reservation Strategies*[1], the authors analyzed the performance of the SLANT algorithm for brain segmentation. They observed that the execution time of SLANT varied significantly depending on the input data, with a walltime of 125min±30%. This result was obtained using a Singularity[6] image of SLANT v1.0 in CPU mode, running on a machine with two Intel Xeon E5-2680v3 processors (12core @ 2,5 GHz) with an unspecified amount of RAM. We can find the exact dataset used to perform the DRD experiments in the git repository associated with the study[5]. This is a dataset containing 87 images of fMRI (Functional magnetic resonance imaging) scans from the Dartmouth Raiders Dataset (DRD)[4]. Each is a 4D scan, with between 326 and 344 pictures at a resolution of 80x80x41 voxels. While SLANT is designed for 3D T1-weighted MRI scans, the DRD dataset works as input for the application, noting that the segmentation output itself is not clinically valid for this data type.

These findings are particularly compelling as they demonstrate significant variability in execution time correlated with input data. Replicating these results would allow us to model and predict this variability based on input parameters, thereby optimizing resource allocation.

To replicate the previous results, we ran the same SLANT CPU v1.0 Singularity implementation on our platform using the same DRD fMRI dataset.

We ran the SLANT algorithm in three different configurations:

- Using 1 thread
- Using 48 threads, with 12 cores per CPU, similarly to the machine used in the previous study
- Using all 128 threads

TABLE I
MEAN AND STANDARD DEVIATION WALLTIME OF SLANT CPU ON SINGULARITY

Nb of threads	Mean time (min)	Std Dev (min)
1	106.33	0.55
48	106.79	4.12
128	108.12	4.11

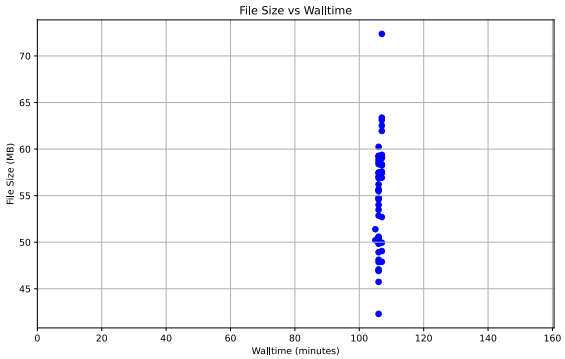

Fig. 2. SLANT CPU Singularity Walltime vs fMRI number of scans for one threads

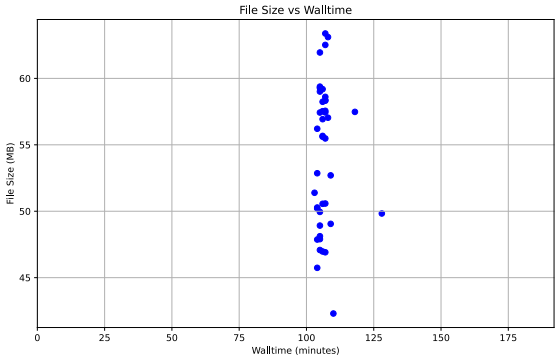

Fig. 3. SLANT CPU Singularity Walltime vs fMRI number of scans for 48 threads

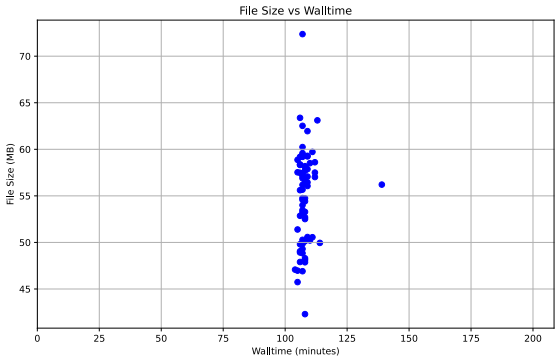

Fig. 4. SLANT CPU Singularity Walltime vs fMRI number of scans for all 128 threads

We observe in Table I a walltime of around 107 minutes regardless of the number of threads used, with a very low standard deviation for all number of threads, with the (still small) highest standard deviation of 48 and 128 threads probably coming from the inter-process communication overhead.

TABLE II
MEAN AND STANDARD DEVIATION WALLTIME OF EACH STEP FOR SLANT CPU ON SINGULARITY (IN SECONDS)

Nb of threads	Prepro-cessing (mean)	Prepro-cessing (std)	Seg-men-tation (mean)	Seg-men-tation (std)	Post-pro-cessing (mean)	Post-pro-cessing (std)
1	931	25	4820	11	588	8
48	836	39	4906	212	660	55
128	856	41	4924	78	705	224

We also decomposed the walltime into its three main steps: preprocessing, segmentation and postprocessing in Table II. Again, we can see that the standard deviation is very low compared to the mean time, showing that the execution time is very consistent regardless of the input. These results are thus very different from the previous study, where the walltime varied significantly depending on the input data.

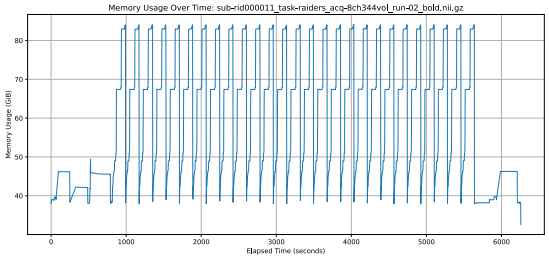

Fig. 5. SLANT CPU Singularity memory profile for all 128 threads

We plotted an example of the memory profile in Fig. 5 obtained when running SLANT CPU on Singularity with all 128 threads. Notice the Y-axis does not start at 0 due to the high background memory usage, however this usage does not impact the performance of the application. We see that the difference between the base memory usage and the peak memory usage is around 42GiB, which is consistent with the previous study. The shape of the memory profile is also very similar, with the preprocessing step consisting of 3 separates peaks, followed by a long segmentation step with 27 peaks, corresponding to the network parameterization of SLANT-27, and finally a postprocessing step with 1 peak.

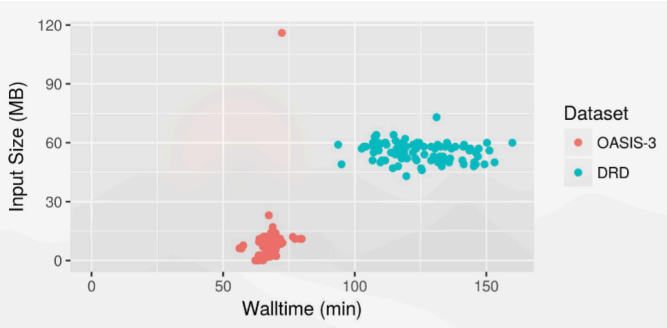

Fig. 6. 2020 study result for SLANT CPU Singularity walltime on two separate datasets

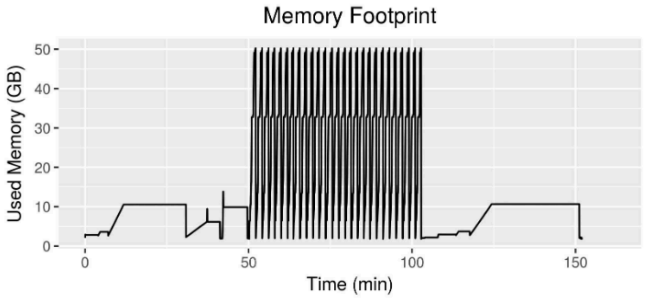

Fig. 7. 2020 study result for SLANT CPU Singularity memory profile on an DRD data

This replication attempt shows that, while we could not replicate the execution time variability observed in the previous study (Fig. 6), we could replicate the memory footprint profile of the SLANT CPU Singularity implementation (Fig. 7). This shows that our platform seem to be running similar jobs as the previous study, but with unidentified system variables leading to a variable time in their case. From our testing with different number of threads, we can also conclude that the time variation in the previous study was not due to the number of threads used.

TABLE III
MEAN AND STANDARD DEVIATION WALLTIME OF EACH STEP FOR SLANT CPU ON DOCKER (IN SECONDS)

Nb of threads	Prepro-cessing (mean)	Prepro-cessing (std)	Seg-men-tation (mean)	Seg-men-tation (std)	Post-pro-cessing (mean)	Post-pro-cessing (std)
128	1324	51	4919	59	1150	44

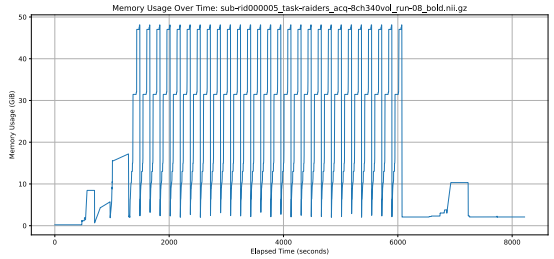

Fig. 8. SLANT CPU Docker memory profile for all 128 threads

In case the paper [1] was using Docker instead of Singularity, we also ran the SLANT CPU Docker in the same 3 configurations. We observe in Table III that the walltime is again very consistent. The mean walltime is a bit higher than the Singularity implementation for the preprocessing and postprocessing phases, but the standard deviation remains very low compared to the mean time for all three steps. Similarly, the memory profile in Fig. 8 is also very similar to the Singularity implementation, with peak usage being a bit higher than the Singularity version, probably due to Docker's overhead. Note that, due to using Docker, we now can plot the absolute memory usage starting at 0.

We can compare the memory profile of SLANT CPU Docker in Fig. 8 with the Singularity version in Fig. 5. We see that the application stays at a very low memory usage at the beginning of the preprocessing step, for around 600 seconds. Similarly, at the beginning and also at the end of the postprocessing step, the memory usage is also very low for around 1000 seconds in total. Altough we do not have an explanation for it, this particular behaviour of Docker could be the source of the higher mean walltime observed in Table III compared to Table II.

This shows that the runtime used (Docker or Singularity) does not impact the non-variability of the execution time we observed, contrary to the previous study. The variability shown in the previous study cannot come from the dual-cpu nature of the platform used, and must thus come from other factors, such as the hardware used or some unknown configuration. We performed the same study on several other machines, and have found again no significant variability in execution time, showing that the variability observed in the previous study is most likely not the normal behaviour of SLANT CPU.

B. Causes of variability and non-variability

In case the previous study used by mistake either a different version or the GPU mode of SLANT, we used the same dataset to run SLANT on the following versions and configurations:

- SLANT v1.0 GPU Docker
- SLANT v1.1 CPU Docker
- SLANT v1.1 GPU Docker

TABLE IV
MEAN AND STANDARD DEVIATION WALLTIME OF SLANT

Mode	Version	Thread count	Mean time (min)	Std Dev (min)
gpu	1.0	128	59.40	1.89
gpu	1.1	128	60.19	3.49
cpu	1.1	128	137.14	1.01

To ensure the discrepancy wasn't due to version mismatches or hardware acceleration differences, we extended our benchmarking to SLANT v1.0 (GPU) and v1.1 (CPU and GPU) in Table IV. Across all configurations, the execution time remained remarkably consistent, reinforcing our conclusion that the software version is not the primary source of the previously observed variability. This consistency persists regardless of the underlying hardware acceleration or specific minor version updates. We also ran SLANT in various versions using a subset of the OASIS-3 T1 image dataset[9], and observed the same non-variability in execution time, showing that the variability observed in the previous study is not due to the specific dataset used, nor due to the resolution of the input images. Thus, we decided to look deeper into the SLANT algorithm itself

The SLANT algorithm is composed of three steps:

1) Preprocessing
2) Segmentation
3) Postprocessing

SLANT relies, for the segmentation part, on a 3D U-Net[10] deep learning model trained on T1-weighted MRI scans. When running SLANT as a user, the segmentation step is performed by using the pre-trained model provided by the authors, without training or fine-tuning. Thus, the segmentation step is deterministic, meaning that for a given input, the output will always be the same. The previous study[1] showed time variations, with the standard deviation being 8.5% of the mean time for the segmentation step, which is substantially higher than our results, but can still be considered low. However, the variation in the previous study is more important in the other steps, with the standard deviation being 25% of the mean time in preprocessing and 50% in postprocessing respectively[1]. These steps seem to be more affected by input variability, which is unexpected compared to our results in Table II. The postprocessing step in particular is finishing with an `antsRegistration` step from the ANTs library[2] , that allows for `1000x1000x1000` maximum steps. This step could be the source of variation of the postprocessing in the previous study, as the number of iterations required for convergence could vary depending on the input data. However, we could not observe this variability in our experiments.

Similarly, the preprocessing step involves a rigid registration that takes almost all the preprocessing time in our runs. This step is performed using `reg_aladin` tool from the NiftyReg library[8]. This tool uses an iterative approach to perform the registration, and the number of iterations required for convergence could vary depending on the input data. Again, we could not observe this variability in our experiments.

III. DEEPMRIPREP

Given the consistent performance of SLANT in our environment, we expanded our scope to `deepmriprep`[7], another prominent neuroscience application. As a deep learning-based MRI preprocessing pipeline, it offers a comparable workload structure but employs different underlying algorithms, allowing us to verify if our observations were specific to SLANT or indicative of a broader trend in containerized neuroimaging tools. The tool can perform multiple preprocessing steps, including:

- brain extraction
- affine registration
- tissue segmentation
- nonlinear registration
- smoothing

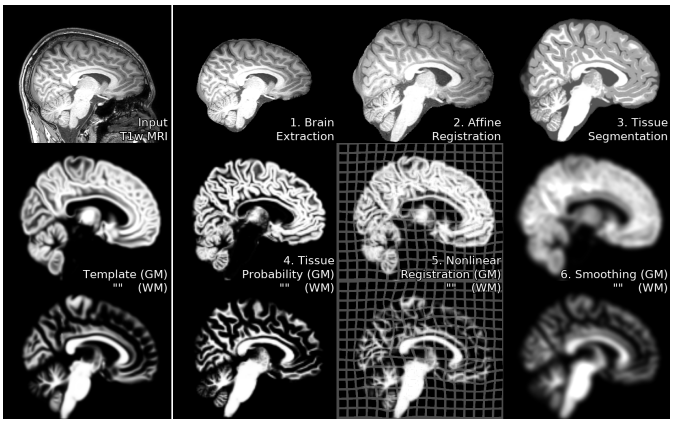

Fig. 9. deepmriprep output example

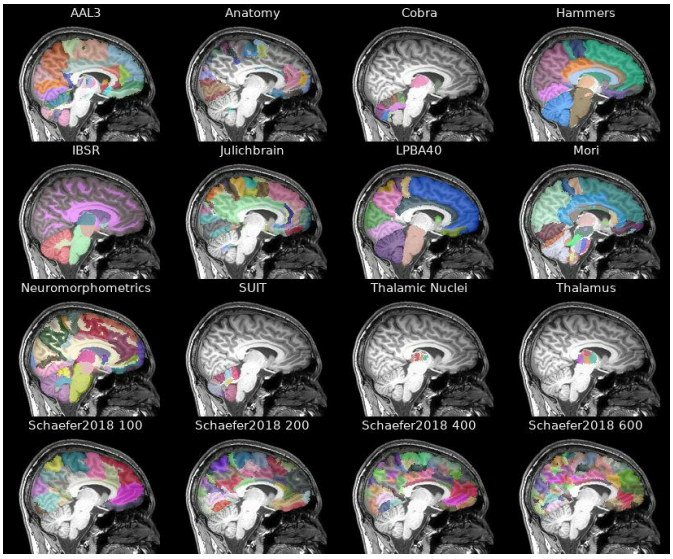

Fig. 10. deepmriprep atlas output example

The application uses a combination of traditional image processing techniques and deep learning models to achieve its results. deepmriprep is a Voxel-Based Morphometry (VBM) tool, meaning that it focuses on analyzing the differences in brain anatomy at the voxel level. However, it can also perform Region-Based Morphometry (RBM) by registering the preprocessed images to a standard atlas. Whether a deep learning model is used depends on the operation being performed.

TABLE V
DEEPMRIPREP PREPROCESSING STEPS AND METHODS USED

Preprocessing step	Method used	Deep learning?
brain extraction	CNN	yes
affine registration	iterative	no
tissue segmentation	3D UNet	yes
nonlinear registration	3D Unet	yes
smoothing	convolution	no

deepmriprep provides three main output types:

- `rbm` (region-based morphometry): standard preprocessing, then atlas registration
- `vbm` (voxel-based morphometry): standard preprocessing, then tiv, mwp1, mwp2, s6mwp1 and s6mwp2
- `all`: perform all preprocessing steps

We ran deepmriprep in CPU mode on our platform using the same fMRI dataset from the Dartmouth Raiders Dataset (DRD). Similarly to SLANT, deepmriprep is expecting 3D, T1-weighted MRI scans as input, but can still process the 4D fMRI scans from the DRD dataset, probably by processing the first 3D scan only. We ran the algorithm using different number of threads (1 and 128) and different output types (`rbm`, `vbm`, and `all`).

TABLE VI
MEAN AND STANDARD DEVIATION WALLTIME OF DEEPMRIPREP IN SECONDS

Type	Nb of threads	Output type	Walltime (mean)	Walltime (std)
cpu	1	all	326.5	2.3
cpu	128	all	85.6	1.6
cpu	128	vbm	75.8	8.14
cpu	128	rbm	51.1	1.0

We observe in Table VI that the walltime is very consistent regardless of the number of threads used or the output type, with a standard deviation being always below 2% of the mean time. This shows that deepmriprep is also not affected by input variability when using the fMRI dataset from DRD, similarly to our study of SLANT.

This result is expected for the deep learning-based steps of deepmriprep, as they are deterministic for a given input. However, some of the other steps, such as affine registration and smoothing, do not use deep learning models and could thus be affected by input variability. The fact that we do not observe any significant variability in execution time suggests that these steps are also not significantly affected by input variability, at least for the fMRI dataset used in this study.

IV. ANTS

Finally, we investigated the Advanced Normalization Tools (ANTs), a widely used open-source software suite for medical image processing and analysis[2]. Unlike the containerized deep learning applications discussed previously, ANTs relies strictly on traditional iterative algorithms for segmentation (specifically Expectation-Maximization). In theory, this algorithmic approach introduces greater potential for runtime variability, as convergence depends heavily on the specific characteristics of the input data. The suite includes:

- image registration
- segmentation
- template building
- brain extraction
- and a lot more

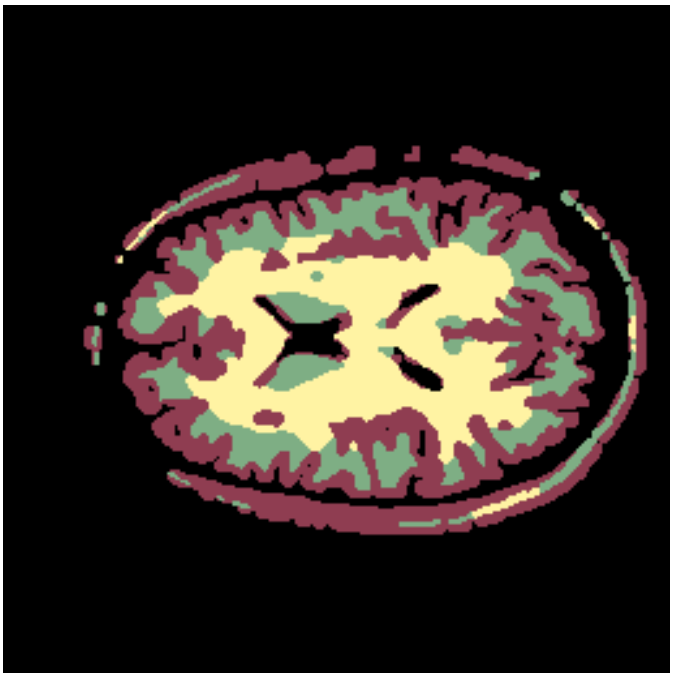

Fig. 11. ANTS atropos output example

We focused our study on the brain segmentation functionality of ANTs, which can be used to segment brain MRI scans into different anatomical regions. ANTs' Atropos algorithm performs segmentation using an iterative approach based on the Expectation-Maximization (EM) algorithm. The algorithm iteratively refines the segmentation by updating the class probabilities and the model parameters until convergence We ran ANTs in CPU mode on our platform but, unlike SLANT and deepmriprep, we used a 3D T1-weighted MRI dataset from the OASIS-3 dataset[9]. The full used dataset metadata can be found in our git repository associated with this study[11], and is composed of 2832 images of T1-weighted MRI scans at variable sizes.

TABLE VII
ATROPOS PARAMETERS

parameter	value
smoothness	0.3,1x1x1
threshold	1e-4
max iterations	50
initialization	kmeans(3)

We ran the Atropos algorithm using the parameters listed in Table VII, which are commonly used for brain segmentation tasks. The most important parameter here is the threshold value, which determines the convergence criteria of the algorithm. The algorithm stops when the log-likelihood change between iterations is below this threshold. This means that the condition, at step n, for stopping the algorithm is:

$$|\text{likelihood}_n - \text{likelihood}_{n-1}| < \text{threshold} \quad (1)$$

It is this treshold value, in combination with the max iteration parameter, that can lead to variability in execution time, as the number of iterations required for convergence can vary depending on the input data. If the threshold is set too low, the algorithm may take a long time to converge, leading to maximum iteration being reached. Conversely, if the threshold is set too high, the algorithm may converge instantly, leading to no variability in execution time. We chose a threshold value of 1e-4, which is a common value used in practice, and a maximum of 50 iterations, which is a higher value than the maximum number of iterations we observed in practice with our dataset.

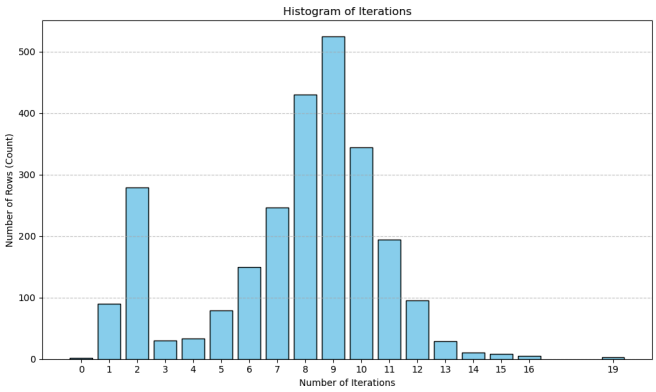

Fig. 12. Atropos histogram of number of iterations to convergence

From the histogram in Fig. 12, we can see that the number of iterations required for convergence varies significantly depending on the input data, with some images converging in one iterations, while others require more than 15 iterations to converge. We can distinguish two distributions shapres in the histogram, with one distribution of images converging in one or two iterations, and another in a normal distribution centered around 9 iterations. To make sure that the variability observed is not purely random, we ran the algorithm multiple times on a subset of 10 images from the dataset in Fig. 13, and observed that the number of iterations required for convergence is roughly the same for each image across different runs, showing that the variability is indeed dependent on the input data.

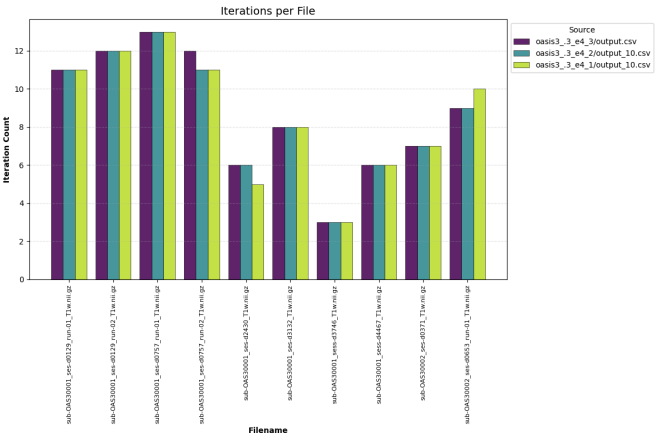

Fig. 13. Atropos number of iterations across 3 different runs

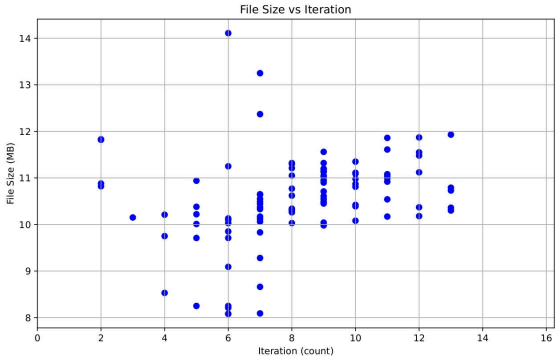

Fig. 14. Atropos number of iterations vs file size

Finally, we plotted the number of iterations required for convergence against the image file size for 100 images of the dataset in Fig. 14, and observed no clear correlation. This suggests that the variability in execution time is not simply due to the size of the input data, but rather to other characteristics of the images that affect the convergence of the algorithm.

A. Deep learning prediction model

We decided to build a prediction model for the iteration count of the Atropos algorithm based on the input image. The model pipeline is as follows:

1) Normalize the intensity of the input image using MONAI's[3] `ScaleIntensity` transform
2) Resize the input image to a fixed size of 64x64x64 voxels using MONAI's `Resize` transform
3) Feature extraction using a classic pattern of `Conv3d` → `BatchNorm` → `ReLU` → `MaxPool`. This 3D CNN pattern is repeated 4 times
4) Regression head consisting of `Flatten` → `Linear` → `ReLU` → `Dropout` → `Linear` to predict the number of iterations required for convergence

We trained the model on a subset of 2000 images and evaluated its performance on a separate test set of 500 images, both from the OASIS-3 dataset[9]. This model took around 3 hours to train on our platform using the GPU and batch mode. We used the Mean Squared Error (MSE) as the loss function for training, and the Mean Absolute Error (MAE) as an additional metric to evaluate the performance of the model. The resulting model has 3,260,929 trainable parameters, with a resulting file size of around 13MB. This is a reasonable size for this task: on average, the model can be loaded in memory in around 3.4s, and the inference time is around 300ms per image, which is negligible compared to the execution time of the Atropos algorithm itself (around 26 seconds for 2-iteration images, and 140 seconds for 14-iteration images).

TABLE VIII
ATROPOS ITERATION COUNT PREDICTION MODEL PERFORMANCE

metric	value
Mean Squared Error (loss)	2.49
Mean Absolute Error	1.14

The model achieved a mean squared error of 2.49 and a mean absolute error of 0.77 on the test set as shown in Table VIII. The Mean Squared Error (MSE) indicates that, on average, the squared difference between the predicted and actual number of iterations is 2.49. This result is a good performance for this task: it shows that it is possible to predict with good precision the number of iterations required for convergence of the Atropos algorithm based on the input image, which could be used to optimize the scheduling of jobs using this algorithm by providing a more accurate walltime estimate.

REFERENCES

[1] Valentin Honoré, Guillaume Pallez Ana Gainaru Brice Goglin. 2021. Profiles of upcoming HPC Applications and their Impact on Reservation Strategies. *IEEE Transactions on Parallel and Distributed Systems* 32, (2021), 1178–1190.

[2] Brian Avants, Nick Tustison, and Gang Song. 2008. Advanced normalization tools (ANTS). *Insight J* (2008), . https://doi.org/10.54294/uvnhin

[3] M. Jorge Cardoso, Wenqi Li, Richard Brown, Nic Ma, Eric Kerfoot, Yiheng Wang, Benjamin Murrey, Andriy Myronenko, Can Zhao, Dong Yang, Vishwesh Nath, Yufan He, Ziyue Xu, Ali Hatamizadeh, Andriy Myronenko, Wentao Zhu, Yun Liu, Mingxin Zheng, Yucheng Tang, Isaac Yang, Michael Zephyr, Behrooz Hashemian, Sachidanand Alle, Mohammad Zalbagi Darestani, Charlie Budd, Marc Modat, Tom Vercauteren, Guotai Wang, Yiwen Li, Yipeng Hu, Yunguan Fu, Benjamin Gorman, Hans Johnson, Brad Genereaux, Barbaros S. Erdal, Vikash Gupta, Andres Diaz-Pinto, Andre Dourson, Lena Maier-Hein, Paul F. Jaeger, Michael Baumgartner, Jayashree Kalpathy-Cramer, Mona Flores, Justin Kirby, Lee A. D. Cooper, Holger R. Roth, Daguang Xu, David Bericat, Ralf Floca, S. Kevin Zhou, Haris Shuaib, Keyvan Farahani, Klaus H. Maier-Hein, Stephen Aylward, Prerna Dogra, Sebastien Ourselin, and Andrew Feng. 2022. MONAI: An open-source framework for deep learning in healthcare. Retrieved from https://arxiv.org/abs/2211.02701

[4] JV Haxby, JS Guntupalli, AC Connolly, YO Halchenko, BR Conroy, MI Gobbini, M Hanke, and PJ Ramadge. 2011. A common, high-dimensional model of the representational space in human ventral temporal cortex. *Neuron* 72, 2 (2011), 404–416. https://doi.org/10.1016/j.neuron.2011.08.026

[5] Valentin Honoré. 2020. Stochastic Application Profiling. Retrieved from https://gitlab.inria.fr/vhonore/stochastic_app_profiling

[6] Bauer MW Kurtzer GM Sochat V. 2017. Singularity: Scientific containers for mobility of compute. *PLoS ONE* 12, 5 (2017).

[7] Janik Goltermann, Carlotta Barkhau, Daniel Emden, Jan Ernsting, Maximilian Konowski, Ramona Leenings, Tiana Borgers, Kira Flinkenflügel, Dominik Grotegerd, Anna Kraus, Elisabeth J. Leehr, Susanne Meinert, Frederike Stein, Lea Teutenberg, Florian Thomas-Odenthal, Paula Usemann, Marco Hermesdorf, Hamidreza Jamalabadi, Andreas Jansen, Igor Nenadic, Benjamin Straube, Tilo Kircher, Klaus Berger, Benjamin Risse, Udo Dannlowski, Tim Hahn Lukas Fisch Nils R. Winter. 2024. deepmriprep: Voxel-based Morphometry (VBM) Preprocessing via Deep Neural Networks. 2024.

[8] Daga P, Winston GP, Duncan JS, Ourselin S Modat M Cash DM. 2014. Global image registration using a symmetric block-matching approach. *J Med Imaging (Bellingham)* (2014). https://doi.org/10.1117/1.JMI.1.2.024003

[9] John C. Morris, Sarah Keefe, Russ Hornbeck, Chengjie Xiong, Elizabeth Grant, Jason Hassenstab, Krista Moulder, Andrei Vlassenko, Marcus E. Raichle, Carlos Cruchaga, Daniel Marcus Pamela J LaMontagne Tammie L.S. Benzinger. 2020. OASIS-3: Longitudinal Neuroimaging, Clinical, and Cognitive Dataset for Normal Aging and Alzheimer Disease. *medRxiv* (2020). https://doi.org/10.1101/2019.12.13.19014902

[10] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-Net: Convolutional Networks for Biomedical Image Segmentation. Retrieved from https://arxiv.org/abs/1505.04597

[11] Axel Vivenot. 2026. Performance Analysis and Walltime Prediction for Neuroscience Applications. Retrieved from https://github.com/seercle/neuro-pds

[12] Katherine Aboud, Parasanna Parvathaneni, Shunxing Bao, Camilo Bermudez, Susan M. Resnick, Laurie E. Cutting, Bennett A. Landman Yuankai Huo Zhoubing Xu. 2018. Spatially Localized Atlas Network Tiles Enables 3D Whole Brain Segmentation. In *International Conference on Medical Image Computing and Computer-Assisted Intervention, MICCAI*, 2018.

[13] Yunxi Xiong, Katherine Aboud, Parasanna Parvathaneni, Shunxing Bao, Camilo Bermudez, Susan M. Resnick, Laurie E. Cutting, Bennett A. Landman Yuankai Huo Zhoubing Xu. 2019. 3D whole brain segmentation using spatially localized atlas network tiles. *NeuroImage* (2019).